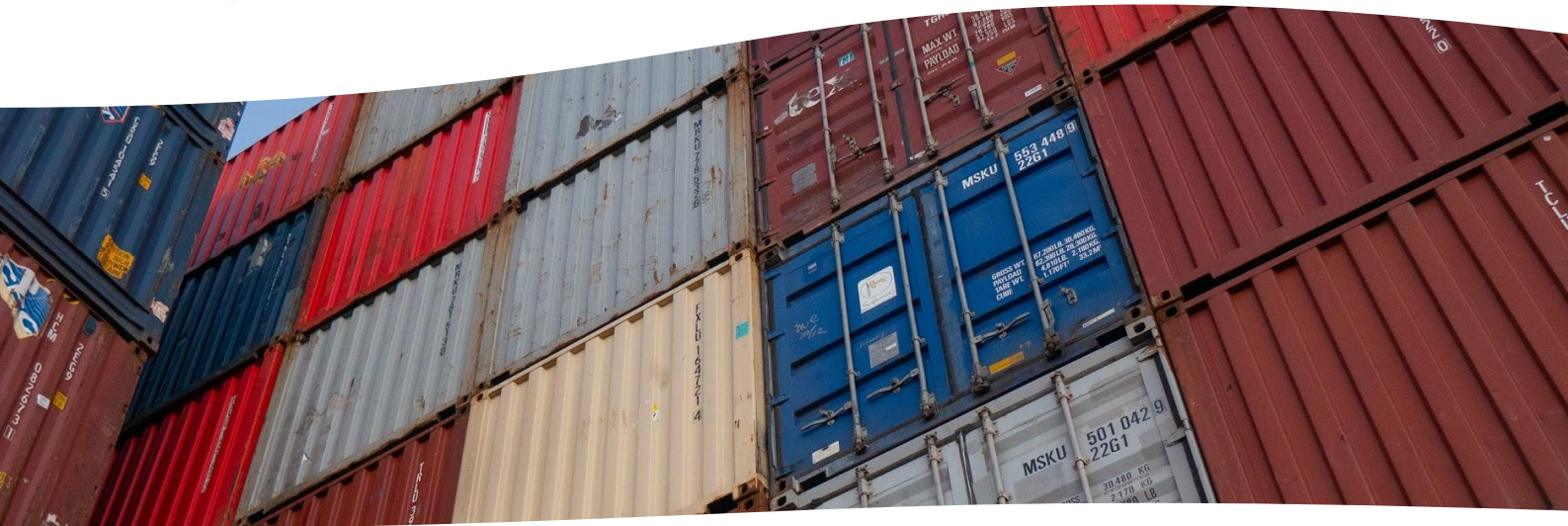


Images et containers avancés



Contexte

DataTune Analytics est une startup spécialisée dans **l'analyse de données musicales**. Nous travaillons avec des plateformes de streaming, des maisons de disques et des artistes pour analyser les tendances d'écoute et optimiser les recommandations.

Aujourd'hui, vous rejoignez notre équipe en tant qu'Analytics Engineer et recevez un mail avec votre première mission de la part de Léo, notre lead data engineer:

"Salut ! Pour bien t'intégrer dans l'équipe, on va commencer par te faire prendre en main notre environnement de travail. Ici, on utilise Docker pour isoler nos applications et faciliter le déploiement. Je vais te confier quelques petites missions pour te faire la main :

Mission 3

Créer un container qui écrit dans un fichier texte accessible après la suppression du container sur la machine hôte – utiliser les Bind Mounts

1. Créer un dossier local de partage nommé shared
2. Lancer un container avec Bind Mounts en mode interactif
3. Écrire 'hello from docker' vers le fichier hello.txt
4. Vérifier que le fichier est lisible sur l'hôte



Mission 4

Créer un container qui écrit dans un fichier texte accessible après la suppression du container sur la machine hôte – en utilisant les Named Volumes cette fois.

Mission 5

Music Converter : Créer un service docker qui convertit un fichier mp4 en mp3. Nous allons créer un container docker qui peut convertir un fichier passé comme argument (ou un fichier par défaut) en utilisant ffmpeg.

La documentation est ici : <https://www.ffmpeg.org/documentation.html>

1. Créer un dossier de stockage local et placer le fichier mp4 fourni dedans.
2. Créer un Dockerfile qui installe le package ffmpeg, place le directory dans /media; gère le bon Entrypoint et une CMD qui target la conversion du fichier.
3. Construire l'image docker.
4. L'exécuter avec un bind mount qui interface le container sans argument (on devrait donc reposer sur l'entrypoint configuré).
5. Vérifier la présence du fichier converti en mp3.
6. Avancé : lancer la même opération mais avec des arguments passés en commande : appelez le résultat sample.mp3.
7. Vérifier la présence de sample.mp3 dans le fichier partagé.
8. *Bonus* : faire de même avec un Named volume à la place de Bind Mounts



Mission 6

Partie 1 – Création du dockerfile

Contexte

Dans un projet data, il est fréquent de devoir publier une documentation technique ou un rapport statique (comme les pages HTML générées par dbt docs).

Pour cela, on peut s'appuyer sur une image Docker personnalisée capable de servir ces fichiers via un petit serveur HTTP intégré.

Objectif

Créer une image Docker capable de servir un fichier HTML (README ou autre documentation) sur le port 8000, en local.

Étapes à suivre

1. Créer un dossier de projet `doc_viewer/` avec :
 - Un fichier `README.html` (simple page HTML)
 - Un Dockerfile pour construire votre image personnalisée
2. Construire une image Docker appelée `doc-viewer`.
3. Exposer le fichier HTML sur le port 8000 grâce à un serveur HTTP Python (`http.server`).
4. Lancer le conteneur Docker en exposant ce port vers l'extérieur, avec `p 8000:8000`.
5. Vérifier dans le navigateur que la documentation est accessible à l'adresse :

<<http://localhost:8000>>

Livrables attendus

- Un dossier `doc_viewer/` complet avec :
 - Dockerfile
 - `README.html`
- Commandes utilisées pour :
 - Construire l'image
 - Lancer le conteneur
- Une capture d'écran de la documentation servie dans le navigateur



Mission 6

Partie 2 – persistance des modifications

On souhaite maintenant s'assurer que cette documentation reste accessible et modifiable même après avoir arrêté ou détruit le conteneur.

Nous allons maintenant utiliser une montée de volume (bind mount) afin que :

les fichiers HTML restent accessibles en dehors du conteneur ;

les modifications effectuées localement (ex: changement de texte, ajout d'un fichier HTML) soient visibles immédiatement dans le navigateur sans avoir à reconstruire l'image ;

la documentation soit persistée au-delà de la durée de vie du conteneur.

Objectif

Mettre en place une solution de persistance simple en Docker, permettant de :

- conserver les fichiers statiques (HTML) sur le système de fichiers hôte ;
- les servir via un serveur HTTP dans le conteneur ;
- éditer et recharger la page sans reconstruire l'image.

